ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO BÀI THỰC HÀNH

Exercise 1: DVC for Data Version Control

23CNTThuc - Ứng dụng NLP trong doanh nghiệp

Giảng viên hướng dẫn:

Nguyễn Hồng Bửu Long

Lương An Vinh

Sinh viên thực hiện:

Trần Phụng Đình (23127527)

Ngày 28 tháng 3 năm 2026

Mục lục

1 Thông tin sinh viên **1**

2 Tổng quan bài thực hành **1**

2.1 Mục tiêu 1

2.2 Giới thiệu về Data Version Control (DVC) 1

3 Data versioning **2**

3.1 Khởi tạo 2

3.2 Kết nối Git repo & DagsHub 3

3.3 Theo dõi phiên bản dữ liệu (Data versioning) 3

4 Retrieve data on another machine **4**

5 Liệt kê và chuyển đổi giữa các phiên bản **7**

5.1 List các phiên bản 7

5.2 Chuyển đổi phiên bản (Switch version) 7

6 Lời cảm ơn **8**

Tài liệu tham khảo **8**

1 Thông tin sinh viên

Họ tên	MSSV	Email
Trần Phụng Đình	23127527	tpdinh23@clc.fitus.edu.vn

2 Tổng quan bài thực hành

2.1 Mục tiêu

Bài thực hành này giúp bạn làm quen với công cụ quản lý phiên bản dữ liệu DVC (Data Version Control) thông qua các nội dung chính:

- Lưu trữ phiên bản dữ liệu (Data versioning).
- Truy xuất dữ liệu trên một thiết bị/máy tính khác (Retrieve data on another machine).
- Liệt kê và chuyển đổi giữa các phiên bản dữ liệu (List/switch between data versions).

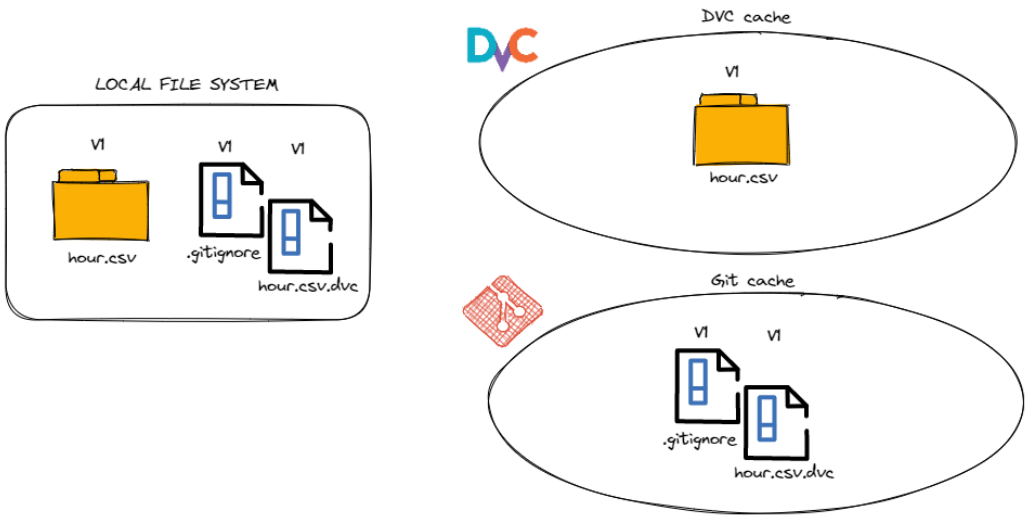
2.2 Giới thiệu về Data Version Control (DVC)

Các công cụ quản lý phiên bản truyền thống như Git thường gặp hạn chế khi lưu trữ dữ liệu có dung lượng lớn. Cụ thể, khi commit một tệp lên Git (ví dụ như trên nền tảng GitHub), dung lượng tối đa cho phép chỉ ở mức 100MB [3]. Nếu vượt quá giới hạn này, người dùng buộc phải chuyển sang sử dụng Git LFS. So với Git LFS, DVC là một giải pháp mã nguồn mở và hoàn toàn miễn phí.

Tiêu chí	**Git LFS**	**DVC**
Bản chất	Tiện ích mở rộng của Git chuyên xử lý tệp lớn.	Công cụ quản lý phiên bản dữ liệu chuyên dụng cho Machine Learning.
Nơi lưu trữ	Yêu cầu máy chủ Git LFS chuyên dụng (thường gắn liền với GitHub/GitLab).	Độc lập với kho lưu trữ (hỗ trợ S3, GCS, Azure, Google Drive, SSH, Local).
Chi phí	Thường bị giới hạn dung lượng/băng thông miễn phí (phải trả thêm tiền nếu vượt mức).	100% miễn phí (bạn chỉ phải trả phí cho dịch vụ lưu trữ Cloud của riêng mình).
Tính năng	Chủ yếu tập trung vào việc lưu trữ và theo dõi các tệp nhị phân lớn.	Hỗ trợ mạnh mẽ việc tạo data pipeline và theo dõi thực nghiệm (experiment tracking).

Bảng 1: So sánh nhanh giữa DVC và Git LFS

Về bản chất, DVC không thực hiện việc quản lý phiên bản hoàn toàn giống như Git. Thay vào đó, nó chỉ tạo ra các tệp siêu dữ liệu (metadata) mang đuôi `.dvc`, hoạt động như là một con trỏ (pointer) [2] để Git có thể theo dõi. Trong khi đó, các tệp dữ liệu dung lượng lớn thực tế sẽ được lưu trữ an toàn bên trong bộ nhớ đệm cục bộ của hệ thống (DVC cache) [1] và đồng bộ với kho lưu trữ từ xa (remote storage).



Hình 1: Cách DVC quản lý phiên bản [1]

Hình 2: Sự khác biệt giữa tệp dữ liệu thực tế và tệp .dvc [1]

3 Data versioning

3.1 Khởi tạo

Trước tiên, chúng ta cần cài đặt thư viện DVC thông qua `pip`:

```
pip install dvc
```

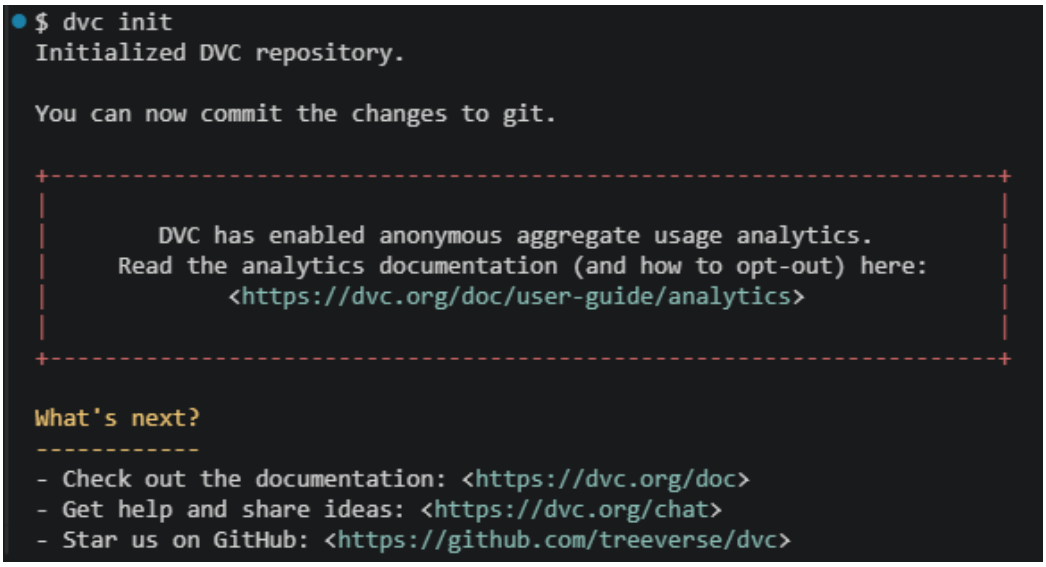
Tiếp theo, khởi tạo một Git repository trống (nếu chưa có):

```
git init
```

Sau đó, tiến hành khởi tạo DVC bên trong dự án Git này:

```
dvc init
```

Sau khi chạy lệnh thành công, màn hình terminal sẽ hiển thị kết quả tương tự như hình dưới đây:



Hình 3: Kết quả sau khi khởi tạo DVC thành công

Lúc này, trong thư mục dự án sẽ tự động xuất hiện thêm thư mục ẩn `.dvc/` cùng với các file `.dvcignore`. Tiếp theo, chúng ta lưu lại (commit) các thay đổi khởi tạo này lên Git:

```
git add .
git commit -m "Initialize Git and DVC"
```

3.2 Kết nối Git repo & DagsHub

Để đẩy code lên GitHub, ta cần thêm cấu hình remote cho Git:

```
git remote add origin https://github.com/tpdinh2612/sentiment-dvc-code.git
```

Đồng thời, để cấu hình nơi lưu trữ dữ liệu (remote storage) cho DVC, ta kết nối với DagsHub bằng lệnh sau:

```
dvc remote add -d dagshub https://dagshub.com/tpdinh2612/sentiment-dvc-code.dvc
```

Sau bước này, bên trong thư mục `.dvc/` sẽ xuất hiện file `config` chứa đường dẫn đến repository trên DagsHub. Tiếp theo, để xác thực (authenticate) tài khoản với DagsHub, ta chạy các lệnh sau:

```
dvc remote modify dagshub auth basic
dvc remote modify dagshub user YOUR_USERNAME
dvc remote modify dagshub password YOUR_TOKEN
```

Lúc này, thông tin xác thực bao gồm tên đăng nhập (username) và token (password) sẽ được tự động lưu trữ an toàn trong file cấu hình `config` của DVC.

3.3 Theo dõi phiên bản dữ liệu (Data versioning)

Để bắt đầu theo dõi một file dữ liệu bằng DVC, ta sử dụng lệnh `add`:

```
dvc add data/raw/sentiment.csv
```

Nếu thực thi thành công, hệ thống sẽ trả về kết quả như sau:

```
$ dvc add data/raw/sentiment.csv
100% Adding...|                                        |1/1 [00:00,  5.01file/s]

To track the changes with git, run:

        git add 'data\raw\sentiment.csv.dvc' 'data\raw\.gitignore'

To enable auto staging, run:

        dvc config core.autostage true
```

Hình 4: Thêm file dữ liệu vào DVC

Bây giờ, trong thư mục `data/raw` sẽ xuất hiện thêm hai file mới là `.gitignore` và `sentiment.csv.dvc`. File `.dvc` này chứa các thông tin siêu dữ liệu (metadata) quan trọng như mã băm (MD5 hash) và kích thước của tập dataset. Mỗi khi dữ liệu gốc có sự thay đổi, ta sẽ lặp lại bước `dvc add`, và file `.dvc` sẽ tự động cập nhật để ghi nhận phiên bản mới.

```
data > raw > sentiment.csv.dvc
1   outs:
    - md5: 345ce3cad2c6e3cb1055c0f52e0702a6
      size: 128
      hash: md5
      path: sentiment.csv
```

Hình 5: File .dvc lưu thông tin metadata của phiên bản dữ liệu

Cuối cùng, tiến hành commit các file cấu hình này lên Git và đẩy đồng thời code lên GitHub, dữ liệu lên DagsHub:

```
git add data/raw/sentiment.csv.dvc data/raw/.gitignore
git commit -m "Track sentiment dataset v1"
git push
```

```
dvc push
```

Sau khi push thành công lên dữ liệu lên Dagshub, sẽ hiển thị thông báo:

```
$ dvc push
Collecting                                             |1.00 [00:00,  141entry/s]
Pushing
1 file pushed
```

Hình 6: File .dvc lưu thông tin metadata của phiên bản dữ liệu

Link Git repository: https://github.com/tpdinh2612/sentiment-dvc-code

4 Retrieve data on another machine

Trong phần này, chúng ta sử dụng nền tảng Google Colab để mô phỏng một session làm việc trên một máy tính khác. Mục đích của việc này là để kiểm chứng khả năng đồng bộ và tải dữ liệu từ kho lưu trữ từ xa (remote storage) về một môi trường cục bộ khác.

Trước tiên, cần cài đặt thư viện DVC cùng với các gói tiện ích mở rộng:

```
!pip install dvc dvc[gdrive]
```

Tiếp theo, clone kho lưu trữ mã nguồn từ GitHub về môi trường hiện tại và di chuyển vào thư mục gốc của dự án:

```
!git clone https://github.com/tpdinh2612/sentiment-dvc-code.git
%cd sentiment-dvc-code/
```

Kết nối với kho dữ liệu trên DagsHub

Để DVC có thể kéo (pull) dữ liệu về máy, hệ thống cần được cấp quyền truy cập vào kho lưu trữ từ xa trên DagsHub. Do kho lưu trữ thường được bảo mật, ta không thể truy cập ẩn danh mà cần có phương thức xác thực.

Để đảm bảo an toàn tuyệt đối, thay vì nhập tài khoản và mật khẩu trực tiếp vào mã nguồn (điều này rất nguy hiểm nếu sổ tay Colab được chia sẻ công khai), chúng ta sẽ tận dụng tính năng **Secrets** của Google Colab để lưu trữ Token bảo mật.
Đoạn mã dưới đây thực hiện việc kết nối với các bước cụ thể như sau:

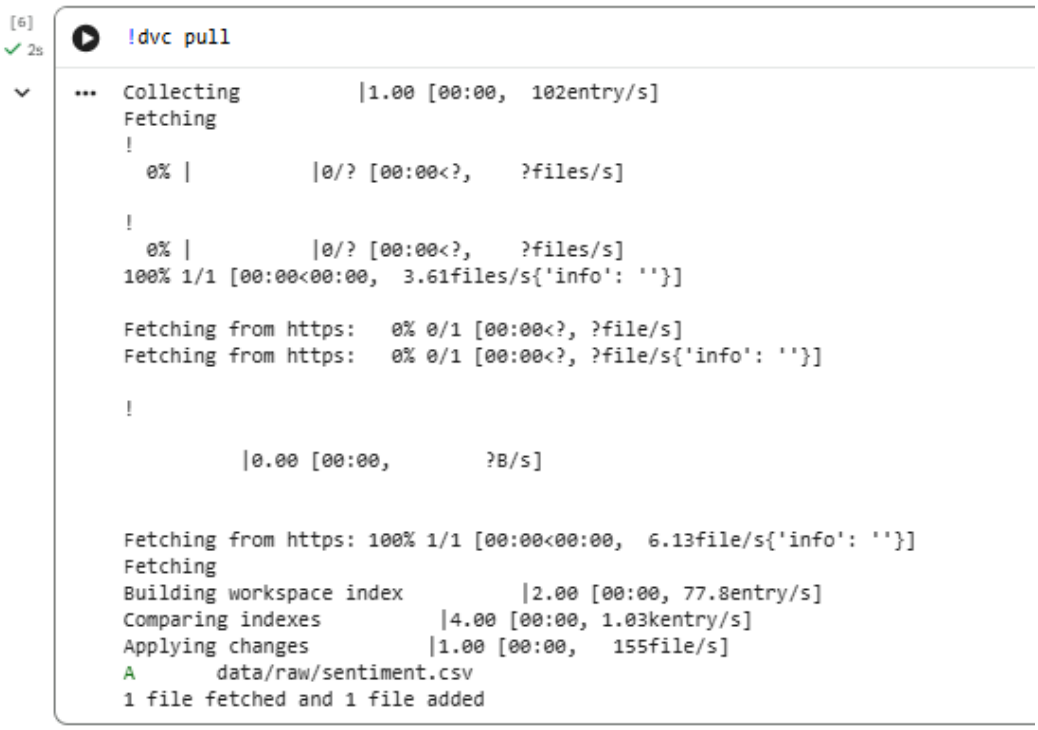
- Truy xuất Token bảo mật từ hệ thống Secrets của Colab.
- Khai báo URL của kho dữ liệu từ xa (remote) trên DagsHub.
- Thiết lập cấu hình bảo mật cục bộ: Các lệnh `dvc remote modify` đi kèm với cờ `-local` sẽ thiết lập xác thực (auth), tên người dùng (user) và mật khẩu (password). Việc sử dụng cờ `-local` là vô cùng quan trọng, nó đảm bảo thông tin đăng nhập chỉ được lưu trong tệp `.dvc/config.local` trên máy ảo hiện tại và sẽ không bao giờ bị đẩy (push) lên Git gây rò rỉ dữ liệu.

```
from google.colab import userdata
import os

# Lay token tu Secrets
dagshub_token = userdata.get('DAGSHUB_TOKEN')

# Dinh nghia URL va them remote
remote_url = "https://dagshub.com/tpdinh2612/sentiment-dvc-data.dvc"
!dvc remote add -d dagshub {remote_url} --force

# Thiet lap cau hinh DVC su dung Token nay (chi luu cuc bo)
!dvc remote modify dagshub --local auth basic
!dvc remote modify dagshub --local user tpdinh2612
!dvc remote modify dagshub --local password {dagshub_token}
```

Sau khi cấu hình xác thực hoàn tất, ta thực hiện lệnh sau để tải dữ liệu về:

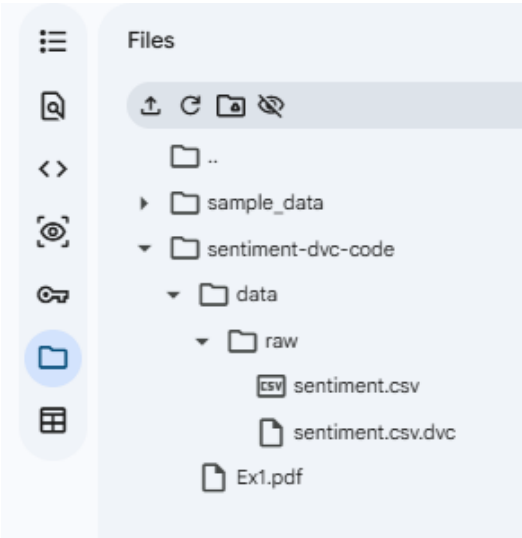
```
!dvc pull
```

Nếu quá trình kết nối và tải dữ liệu diễn ra thành công, màn hình sẽ hiển thị kết quả tương tự như sau:



Hình 7: Thông báo thành công khi chạy lệnh dvc pull

Lúc này, khi kiểm tra trong cây thư mục của Colab, tại đường dẫn `data/raw` sẽ xuất hiện tệp `sentiment.csv`:

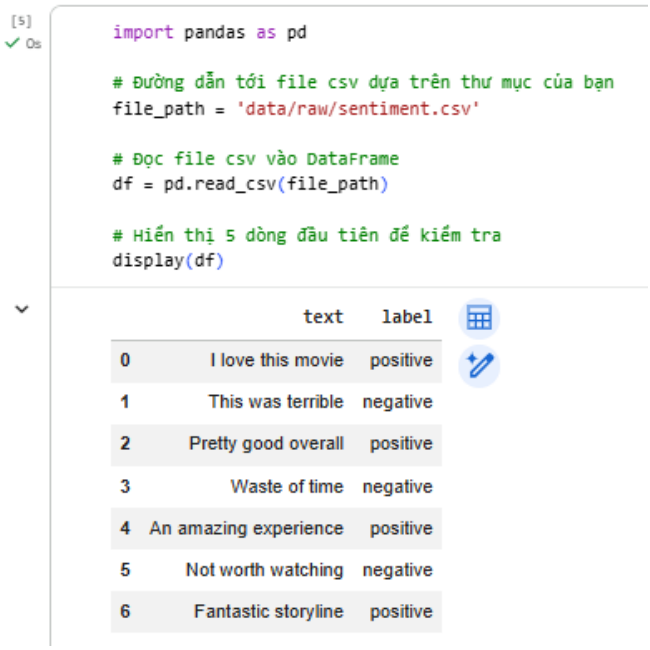


Hình 8: Tệp dữ liệu xuất hiện trong thư mục data/raw

Ta có thể chạy một đoạn mã Python nhỏ sử dụng thư viện Pandas để đọc thử file:

```
import pandas as pd

file_path = 'data/raw/sentiment.csv'
df = pd.read_csv(file_path)
display(df)
```

Kết quả hiển thị khung dữ liệu (DataFrame) xác nhận rằng quy trình đồng bộ hóa qua DVC để lấy dữ liệu mới nhất đã thành công:



```
[5]
import pandas as pd

# Đường dẫn tới file csv dựa trên thư mục của bạn
file_path = 'data/raw/sentiment.csv'

# Đọc file csv vào DataFrame
df = pd.read_csv(file_path)

# Hiển thị 5 dòng đầu tiên để kiểm tra
display(df)
```

	text	label
0	I love this movie	positive
1	This was terrible	negative
2	Pretty good overall	positive
3	Waste of time	negative
4	An amazing experience	positive
5	Not worth watching	negative
6	Fantastic storyline	positive

Hình 9: Kết quả đọc và hiển thị dữ liệu từ tệp sentiment.csv

5 Liệt kê và chuyển đổi giữa các phiên bản

5.1 List các phiên bản

Để xem lịch sử các lần commit liên quan đến tệp `.dvc` của dữ liệu, ta sử dụng lệnh sau [2]:

```
!git log --oneline data/raw/sentiment.csv.dvc
```

Kết quả sẽ hiển thị danh sách các lần commit như hình dưới đây:

```
d0526f2 (HEAD -> master, origin/master, origin/HEAD) Dataset v2: add more rows
76527e6 Track sentiment dataset v1
```

Hình 10: Lịch sử các lần commit của tệp dữ liệu

5.2 Chuyển đổi phiên bản (Switch version)

Để chuyển về một phiên bản dữ liệu cụ thể, đầu tiên ta cần thay thế mã hash của commit tương ứng:

```
hash_commit = "76527e6"
!git checkout {hash_commit} data/raw/sentiment.csv.dvc
```

Như đã đề cập ở phần 2.2, Git chỉ quản lý và lưu trữ metadata bao gồm các tệp mã nguồn và tệp `.dvc`. Git hoàn toàn không chứa các tệp dữ liệu lớn thực tế.

Thay vào đó, DVC đảm nhận việc lưu trữ dữ liệu lớn thực tế (dataset, các tệp mô hình như `.pt`, `.h5`,...) bên trong thư mục bộ nhớ đệm cục bộ (`.dvc/cache`) hoặc trên các máy chủ lưu trữ từ xa (Remote Storage như S3, Google Drive, DagsHub,...).

Do đó, sau khi thực hiện lệnh `git checkout` thành công, Git chỉ mới cập nhật nội dung của tệp `.dvc`. Ta cần chạy thêm các lệnh của DVC để tải dữ liệu thực tế từ Remote Storage:

```
!dvc pull
!dvc checkout
```

Sau khi hoàn tất, ta có thể in thử để kiểm tra lại dữ liệu:

Hình 11: Kiểm tra dữ liệu sau khi chuyển đổi phiên bản

Như vậy, dữ liệu đã được switch thành công về phiên bản trước đó!

Link Google Colab notebook: Ex1 - DVC Data Version Control.ipynb

6 Lời cảm ơn

Em xin chân thành cảm ơn các thầy **Nguyễn Hồng Bửu Long, Lương An Vinh** đã hướng dẫn bộ môn để em có kiến thức nền tảng hoàn thành bài thực hành này. Bài thực hành này được em tự thực hiện dựa trên các kiến thức đã học và các tài liệu tham khảo. Trong quá trình thực hiện đồ án, em có sử dụng công cụ AI Gemini để hỗ trợ và chỉnh sửa câu từ, đảm bảo report được trình bày một cách rõ ràng hơn.

Tài liệu

[1] Dagshub blog, getting started with data version control (dvc). https://dagshub.com/blog/getting-started-with-dvc/, 2024. Truy cập ngày: 28/03/2026.

[2] Datacamp, data version control (dvc) tutorial. https://www.datacamp.com/tutorial/data-version-control-dvc, 2024. Truy cập ngày: 28/03/2026.

[3] Github docs, about large files on github. https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github, 2024. Truy cập ngày: 28/03/2026.